

SynthDoc

AI Document Generator

Java / Javalin 6.6.0 / Gradle (Groovy DSL)

Report Date: 2026-03-10

1. Project Overview

SynthDoc is an AI Document Generator built with Java and the Javalin 6.6.0 web framework. It provides document templates with AI-powered content completion, citation tracking, document versioning, and export capabilities. The system uses a mocked Claude API client to enable fully deterministic testing without real API keys.

Language:	Java (JDK 21 toolchain, built on JDK 25)
Framework:	Javalin 6.6.0
Build Tool:	Gradle 9.4.0 (Groovy DSL)
Testing:	JUnit 5.11.0
Storage:	ConcurrentHashMap (in-memory)
Serialization:	Jackson 2.18.0

2. Architecture

Models (6 classes)

- Template - Document template with type, sections, and variables
- Section - Document section with type (introduction/body/conclusion/references/custom)
- Document - Document instance with sections, metadata, and version tracking
- DocumentVersion - Immutable snapshot of a document at a point in time
- Citation - Citation with format support (APA/MLA/Chicago)
- GenerationRequest - AI generation request with tone and context parameters

Services (7 classes)

- TemplateService - CRUD operations for document templates
- DocumentService - Document lifecycle management with template instantiation
- GenerationService - AI content generation using Claude client abstraction
- CitationService - Citation management and bibliography generation
- VersionService - Version history, rollback, and diff capabilities
- ExportService - Export documents as JSON, text, or markdown
- StatsService - Platform-wide statistics and metrics

Clients (3 classes)

- ClaudeClient - Interface defining the AI generation contract
- HttpClient - Real implementation using `java.net.http.HttpClient`
- MockClaudeClient - Deterministic mock for testing with custom response registration

3. API Endpoints

Method	Endpoint	Description
GET	/health	Health check
POST	/api/templates	Create template
GET	/api/templates	List all templates
GET	/api/templates/{id}	Get template by ID
PUT	/api/templates/{id}	Update template
DELETE	/api/templates/{id}	Delete template
POST	/api/documents	Create document from template
GET	/api/documents	List all documents
GET	/api/documents/{id}	Get document by ID
POST	/api/documents/{id}/generate	Generate all sections
POST	/api/documents/{id}/sections/{sid}/generate	Generate single section
GET	/api/documents/{id}/versions	Get version history
POST	/api/documents/{id}/rollback/{ver}	Rollback to version
GET	/api/documents/{id}/export?format=X	Export document
POST	/api/citations	Create citation
GET	/api/documents/{id}/citations	Get document citations
GET	/api/documents/{id}/bibliography	Generate bibliography
GET	/api/stats	Platform statistics

4. Test Results

All 117 tests pass successfully across 8 test suites:

Test Suite	Tests	Status
TemplateServiceTest	16	PASSED
DocumentServiceTest	18	PASSED
GenerationServiceTest	10	PASSED
CitationServiceTest	15	PASSED
VersionServiceTest	14	PASSED
ExportServiceTest	16	PASSED
ClaudeClientTest	12	PASSED
ApiIntegrationTest	16	PASSED
TOTAL	117	ALL PASSED

5. Security Audit (10-Point)

1. Secrets and API Keys

PASS - No hardcoded API keys or secrets. CLAUDE_API_KEY read from environment variable only. MockClaudeClient used for all tests.

2. Input Validation

PASS - All service methods validate required fields (name, type, title, source). IllegalArgumentException thrown for invalid inputs.

3. SQL Injection / NoSQL Injection

N/A - No database used. ConcurrentHashMap storage eliminates injection vectors.

4. Cross-Site Scripting (XSS)

LOW RISK - JSON-only API. No HTML rendering. Jackson handles output encoding. Client-side rendering responsibility.

5. Authentication and Authorization

NOTE - No authentication implemented. Suitable for internal/demo use. Production deployment should add API key or JWT authentication.

6. Dependency Security

PASS - Using latest stable versions: Javalin 6.6.0, Jackson 2.18.0, JUnit 5.11.0. No known CVEs in dependencies.

7. Error Handling and Information Leakage

PASS - Custom exception handlers return sanitized error messages. Stack traces not exposed to clients.

8. Concurrency Safety

PASS - ConcurrentHashMap for all stores. Thread-safe operations. No race conditions in version management.

9. Docker Security

PASS - Multi-stage build minimizes image size. Non-root user (appuser). Alpine-based JRE image reduces attack surface.

10. CI/CD Pipeline Security

PASS - GitHub Actions with pinned action versions (v4). No secrets in workflow. Docker build verification in CI pipeline.

6. Build Errors and Resolutions

Error 1: JUnit Platform Launcher Missing

Gradle 9.4 with JDK 25 requires explicit JUnit Platform Launcher dependency. Resolved by adding `testRuntimeOnly 'org.junit.platform:junit-platform-launcher'` to `build.gradle` dependencies.

Error 2: Jackson Unknown Field 'duplex'

OkHttp RequestBody includes internal metadata fields that Jackson tried to deserialize into Template objects. Resolved by configuring ObjectMapper with `DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES` disabled.

Error 3: API Test Client Compatibility

Javalin TestTools OkHttp client caused request body serialization issues. Resolved by rewriting API integration tests to use `java.net.http.HttpClient` with explicit Content-Type headers for clean JSON transport.

7. Technology Stack

Component	Technology
Runtime	Java 21 (toolchain) on JDK 25
Web Framework	Javalin 6.6.0
Build Tool	Gradle 9.4.0 (Groovy DSL)
JSON	Jackson 2.18.0 + JSR310 module
Testing	JUnit 5.11.0
Logging	SLF4J Simple 2.0.16
AI Client	Claude API (mocked for tests)
Storage	ConcurrentHashMap (in-memory)
Container	Docker (eclipse-temurin:21-jre-alpine)
CI/CD	GitHub Actions